Server.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <time.h>

#define PORT 8080

int main() {
   int sockfd;
   char buffer[1024];
   struct sockaddr_in servaddr, cliaddr;

   sockfd = socket(AF_INET, SOCK_DGRAM, 0);

   servaddr.sin_family = AF_INET;
   servaddr.sin_addr.s_addr = INADDR_ANY;
   servaddr.sin_port = htons(PORT);


   bind(sockfd, (const struct sockaddr *)&servaddr, sizeof(servaddr));

   printf("Concurrent Time Server (UDP) ready on port %d...\n", PORT);

   while(1) {
      socklen_t len = sizeof(cliaddr);
      recvfrom(sockfd, buffer, 1024, 0, (struct sockaddr *)&cliaddr, &len);

      if(fork() == 0) {

         time_t t = time(NULL);
         char *time_str = ctime(&t);
         sendto(sockfd, time_str, strlen(time_str), 0, (const struct sockaddr *)&cliaddr, len);
         exit(0);
      }
   }
   return 0;
}
```

Client.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <time.h>

#define PORT 8080

int main() {
    int sockfd;
    char buffer[1024];
    struct sockaddr_in servaddr, cliaddr;

    sockfd = socket(AF_INET, SOCK_DGRAM, 0);

    servaddr.sin_family = AF_INET;
    servaddr.sin_port = htons(PORT);
    inet_pton(AF_INET, "127.0.0.1", &servaddr.sin_addr);

    char *msg = "TIME_REQUEST";
    printf("Sending time request to server...\n");
    sendto(sockfd, msg, strlen(msg), 0, (const struct sockaddr *)&servaddr, sizeof(servaddr));
    int len = sizeof(servaddr);
    int n = recvfrom(sockfd, buffer, 1024, 0, (struct sockaddr *)&servaddr, &len);
    if (n >= 0) {
        buffer[n] = '\0';
    }
    printf("Server System Time: %s\n", buffer);
    close(sockfd);
    return 0;
}
```

OUTPUT



